dimensional (squared) distances, and each requires `O(d)` operations to be computed;

- Comparing two points classifications requires `O(n)` time, but this can be amortized inside the method doing the partitioning, with a careful implementation;
- Updating the centroids requires computing, for each centroid, `d` times (one for each coordinate[9]) a mean over at most `n` points, and so a total of `O(k*n*d)` operations. If we can assume that the points are distributed evenly among clusters, each cluster will at most contain `n/k` points, and therefore the average worst-case running time becomes `O(k*(n/k)*d) = O(n*d)`.

The running time for the algorithm is therefore `O(m*k*n*d)`, with `O(n+k)` extra memory needed to store the points classification and the list of centroids.

12.2.4 Boosting k-means with k-d Trees

When we described the code for k-means, we saw that the partitioning step, where we assign each point to exactly one of the centroids, is a brute-force search among on all combinations of points and centroids; now we wonder: we can speed it up in any way?

In section 10.5.3 we have seen how k-means can help Ss^+-trees' performance by making them more balanced, when used in split heuristics. But... is it possible that the opposite is also true? You might see where I'm going with this: is it